

JavaScript Execution Model & Event Loop — অ্যাসিঙ্ক আর্কিটেকচার শেখার মূল উদ্দেশ্য

JavaScript সিঙ্গেল-থ্রেডেড হয়েও কীভাবে বিলিয়ন রিকোয়েস্ট, নেটওয়ার্ক আই/ও এবং নন-ব্লকিং ইউআই পরিচালনা করে—তার পেছনের চালিকাশক্তি হলো **Event Loop ও Execution Context**। এই চ্যাপ্টারে আমরা **Call Stack (LIFO)**, **Memory Heap**, **Web APIs**, **Microtask Queue (Promises)**, **Macrotask Queue (Timers)** এবং ব্রাউজার রেন্ডারিং পাইপলাইন গভীরভাবে শিখব।

- 1. Execution Model
- 2. Single-Threaded Nature
- 3. Runtime Components
- 4. V8 Engine Architecture
- 5. Call Stack (LIFO)
- 6. Stack Overflow Trap
- 7. Memory Heap Allocations
- 8. Execution Context (GEC/FEC)
- 9. Creation & Execution Phase
- 10. Lexical Scope Chain
- 11. Browser Host Web APIs
- 12. Event Loop Core Algorithm
- 13. Task / Macrotask Queue
- 14. Microtask Queue Priority
- 15. setTimeout(fn, 0) Reality
- 16. Promise Microtask Drain
- 17. Async/Await Execution Order
- 18. fetch() & DOM Queues
- 19. Microtask Starvation
- 20. Rendering Frame Sync
- 21. queueMicrotask() API
- 22. React Concurrent Mode
- 23. Node.js Event Loop Phases
- 24. 4 Execution Trace Labs

Part 01 — JavaScript Engine, Call Stack & Memory Heap (29.1 – 29.10)

29.1 JavaScript Execution Model কী?

JavaScript code কীভাবে: 1. Read করে 2. Execute করে 3. Function call handle করে 4. Asynchronous কাজ manage করে 5. পরে result execute করে —এই পুরো system-টাই JavaScript-এর **Execution Model**। সহজভাবে:

ARCHITECTURE / CONCEPT FLOW

```
JavaScript Code
↓
Execution Context
↓
Call Stack
↓
Synchronous Execution
↓
Async কাজ → Host APIs
↓
Task / Microtask Queue
↓
Event Loop
↓
Call Stack
↓
Execution
```

29.2 JavaScript কি Single-Threaded?

সাধারণ JavaScript execution model-এ একটি main JavaScript execution thread-এ এক সময়ে **একটি piece of JavaScript code** execute হয়। তাই আমরা বলি: > JavaScript is single-threaded. উদাহরণ:

```
JAVASCRIPT SOURCE JAVASCRIPT
console.log("A");
console.log("B");
console.log("C");
```

Output:

```
► CONSOLE OUTPUT OUTPUT
A
B
C
```

একসাথে:

```
► CONSOLE OUTPUT OUTPUT
A
B
C
```

execute হয় না। একটির execution শেষ হওয়ার পর পরেরটি execute হয়।

◆ কিন্তু একটা গুরুত্বপূর্ণ বিষয়

এর মানে এই নয় যে browser একেবারেই parallel কাজ করতে পারে না। Browser/Node.js-এর মতো **host environment** JavaScript-এর বাইরে বিভিন্ন কাজ handle করতে পারে—যেমন:

- Timer
- Network request
- File I/O
- DOM events
- Background operations

এবং JavaScript পরে সেই কাজের result handle করে।

29.3 JavaScript Runtime-এর Main Components

একটি simplified browser JavaScript runtime:



এগুলো একে একে বুঝি।

29.4 JavaScript Engine

JavaScript Engine হলো সেই software system যেটা JavaScript code বুঝে execute করে। Popular engines:

Environment	Engine
Chrome	V8
Edge	V8
Node.js	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

উদাহরণ:

```
JAVASCRIPT SOURCE JAVASCRIPT

const x = 10;

console.log(x);
```

Engine এই code process করে execute করে।

29.5 Call Stack কী?

Call Stack হলো JavaScript-এর function execution track করার একটি mechanism। সহজভাবে: > কোন function এখন execute হচ্ছে এবং কোন function থেকে call হয়েছে— Call Stack সেটা track করে। উদাহরণ:

```
JAVASCRIPT SOURCE JAVASCRIPT

function one() {
  two();
}

function two() {
  three();
}

function three() {
  console.log("Hello");
}

one();
```

Execution:

```
🕒 ARCHITECTURE / CONCEPT FLOW

one()
↓
two()
↓
three()
```

Call Stack:

```
🕒 ARCHITECTURE / CONCEPT FLOW

three()
two()
one()
```

three() শেষ হলো:

```
▶ CONSOLE OUTPUT OUTPUT

three() removed
```

তারপর:

```
▶ CONSOLE OUTPUT OUTPUT

two() removed
```

তারপর:

```
▶ CONSOLE OUTPUT OUTPUT

one() removed
```

29.6 Call Stack — Real Life Example

ধরো তুমি একটি restaurant-এ order দিচ্ছে।

🕒 ARCHITECTURE / CONCEPT FLOW

```
Customer
↓
Order
↓
Food preparation
```

যে কাজটি এখন চলছে সেটা top-এ থাকবে। একইভাবে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
main()
↓
login()
↓
validateUser()
```

সবচেয়ে নিচে:

▶ CONSOLE OUTPUT

OUTPUT

```
main()
```

সবচেয়ে উপরে:

▶ CONSOLE OUTPUT

OUTPUT

```
validateUser()
```

29.7 Stack হলো LIFO

Call Stack সাধারণত **LIFO** principle follow করে। > Last In, First Out উদাহরণ:

🕒 ARCHITECTURE / CONCEPT FLOW

```
A()
↓
B()
↓
C()
```

প্রথমে C() শেষ হবে।

▶ CONSOLE OUTPUT

OUTPUT

```
C()
B()
A()
```

29.8 Simple Call Stack Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("Start");

function hello() {
  console.log("Hello");
}

hello();

console.log("End");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Start
Hello
End
```

Execution:

🕒 ARCHITECTURE / CONCEPT FLOW

```
console.log("Start")
  ↓
hello()
  ↓
console.log("Hello")
  ↓
console.log("End")
```

29.9 Stack Overflow

যদি function নিজেকেই continuously call করে এবং কোনো stopping condition না থাকে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function test() {
  test();
}

test();
```

তাহলে:

► CONSOLE OUTPUT

OUTPUT

```
test()
test()
test()
test()
...
```

Call Stack পূর্ণ হয়ে যাবে। শেষে error:

► CONSOLE OUTPUT

OUTPUT

```
RangeError: Maximum call stack size exceeded
```

এটাকে বলে: > **Stack Overflow**

29.10 Heap কী?

Heap হলো memory-এর একটি area যেখানে JavaScript runtime dynamically allocated data রাখে। যেমন:

```
JAVASCRIPT SOURCE JAVASCRIPT

const user = {
  name: "Ripon",
  age: 23
};
```

Object-এর data runtime memory-তে রাখা হয়। Array:

```
JAVASCRIPT SOURCE JAVASCRIPT

const numbers = [10, 20, 30];
```

Function/object-এর মতো reference-based data heap-এর সাথে সম্পর্কিত। সহজভাবে:

```
🕒 ARCHITECTURE / CONCEPT FLOW

Call Stack
→ execution tracking

Heap
→ dynamically allocated objects/data
```

Memory Management বিস্তারিতভাবে আমরা **Chapter 30**-এ দেখব।

Part 02 — Execution Context, Lifecycle Phases & Scope Chain (29.11 – 29.17)

29.11 Execution Context কী?

JavaScript যখন code execute করে, তখন তার জন্য একটি **Execution Context** তৈরি হয়। সহজভাবে: > JavaScript code execute করার জন্য যে environment তৈরি হয় সেটাই Execution Context। প্রধানত:

♦ 1. Global Execution Context

সবচেয়ে প্রথমে তৈরি হয়।

♦ 2. Function Execution Context

প্রতিবার function call হলে তৈরি হয়।

♦ 3. Eval Execution Context

`eval()` ব্যবহার করলে তৈরি হতে পারে। Modern development-এ সাধারণত avoid করা হয়।

29.12 Global Execution Context

এই code:

```
JAVASCRIPT SOURCE JAVASCRIPT

const name = "Ripon";

console.log(name);
```

execute করার সময় JavaScript একটি:

```
🕒 ARCHITECTURE / CONCEPT FLOW

Global Execution Context
```

তৈরি করে। এখানে global-level variables/functions এবং অন্যান্য execution information থাকে।

29.13 Function Execution Context

এই code:

```
JAVASCRIPT SOURCE JAVASCRIPT

function greet(name) {
  console.log(name);
}

greet("Ripon");
```

`greet()` call হলে নতুন Function Execution Context তৈরি হয়।

```
🕒 ARCHITECTURE / CONCEPT FLOW

Global Execution Context
  ↓
Function Execution Context
  ↓
greet()
```

`greet()` শেষ হলে function context চলে যায়।

29.14 Execution Context-এর Creation Phase

একটি execution context তৈরি হওয়ার সময় JavaScript আগে execution-এর জন্য environment প্রস্তুত করে। সহজভাবে:

```
🕒 ARCHITECTURE / CONCEPT FLOW

Creation Phase
  ↓
Environment তৈরি
  ↓
Declarations setup
  ↓
Execution Phase
  ↓
Code execute
```

এই কারণে আমরা **hoisting** দেখতে পাই। উদাহরণ:

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log(x);

var x = 10;
```

Output:

```
▶ CONSOLE OUTPUT OUTPUT

undefined
```

কারণ `var x` declaration আগে থেকেই environment-এ setup হয়। তবে:

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log(x);

let x = 10;
```

এতে error:

```
▶ CONSOLE OUTPUT OUTPUT

ReferenceError
```

কারণ `let` এবং `const` declaration execution-এর আগে scope-এ থাকলেও initialization-এর আগে access করা যায় না। এই সময়টাকে **Temporal Dead Zone (TDZ)** বলা হয়।

29.15 Execution Phase

তারপর actual code execute হয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const x = 10;

console.log(x);
```

Conceptually:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Creation Phase
  ↓
x environment-এ setup
  ↓
Execution Phase
  ↓
x = 10
  ↓
console.log(x)
```

29.16 Lexical Environment

JavaScript variables কোথায় থাকবে এবং কোন variable কোথা থেকে accessible হবে—এটার সাথে **Lexical Environment** সম্পর্কিত। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const globalName = "Ripon";

function greet() {
  const message = "Hello";

  console.log(globalName);
  console.log(message);
}

greet();
```

`greet()` access করতে পারে:

► CONSOLE OUTPUT

OUTPUT

```
message
globalName
```

কারণ scope chain-এর মাধ্যমে outer environment পাওয়া যায়।

29.17 Scope Chain

JAVASCRIPT SOURCE

JAVASCRIPT

```
const a = 10;

function outer() {
  const b = 20;

  function inner() {
    const c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }

  inner();
}

outer();
```

`inner()` দেখতে পারে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
c → নিজের scope
b → outer scope
a → global scope
```

এটা scope chain-এর basic ধারণা। Closure-এর সাথে এই বিষয়টি খুব গুরুত্বপূর্ণ—যেটা Chapter 26-এ আমরা বিস্তারিত করেছি।

29.18 Synchronous JavaScript

Synchronous code line-by-line execute হয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("1");  
console.log("2");  
console.log("3");
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
1  
2  
3
```

Flow:

🕒 ARCHITECTURE / CONCEPT FLOW

```
1  
↓  
2  
↓  
3
```

29.19 Asynchronous JavaScript কেন দরকার?

ধরো server থেকে data আনতে 5 seconds লাগছে। যদি JavaScript পুরো thread block করে অপেক্ষা করত:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Request  
↓  
Wait 5 seconds  
↓  
Continue
```

তাহলে webpage freeze হয়ে যেত। তাই browser/host environment asynchronous APIs ব্যবহার করে।

29.20 Web APIs কী?

Browser JavaScript language-এর নিজস্ব অংশ নয়, কিন্তু browser কিছু APIs দেয়। যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setTimeout()  
fetch()  
document.querySelector()  
addEventListener()  
localStorage
```

এগুলো browser/host environment-এর capabilities-এর সাথে কাজ করে। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setTimeout(() => {  
  console.log("Hello");  
}, 2000);
```

Timer-এর কাজ host environment handle করে।

29.21 Event Loop কী?

Event Loop হলো এমন একটি mechanism যা দেখে: > Call Stack কখন empty হয়েছে এবং queued asynchronous callbacks কখন JavaScript-এর execution-এর জন্য Stack-এ পার্তানো যাবে। Simplified model:



29.22 Task Queue / Macrotask Queue

Task queue-তে বিভিন্ন asynchronous callback অপেক্ষা করতে পারে। উদাহরণ:

JAVASCRIPT SOURCE JAVASCRIPT

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);
```

Timer-এর callback ready হলে task queue-তে যেতে পারে। আরও উদাহরণ:

- `setTimeout`
- `setInterval`
- কিছু DOM events
- অন্যান্য host-scheduled tasks

29.23 Microtask Queue

Microtask Queue সাধারণত asynchronous callbacks-এর জন্য ব্যবহৃত হয় যেগুলো current task শেষ হওয়ার পর দ্রুত execute হওয়ার জন্য schedule হয়। সবচেয়ে গুরুত্বপূর্ণ:

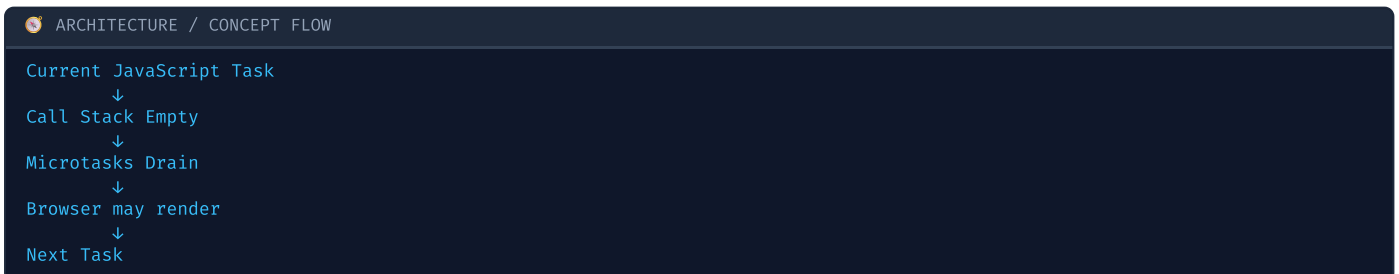
JAVASCRIPT SOURCE JAVASCRIPT

```
Promise.then()  
Promise.catch()  
Promise.finally()  
queueMicrotask()
```

Browser-এ `MutationObserver` -ও microtask mechanism ব্যবহার করে।

29.24 সবচেয়ে গুরুত্বপূর্ণ Rule 🔥

Simplified browser model:



অর্থাৎ: > **Microtask সাধারণত next task-এর আগে execute হয়।** এটা Event Loop-এর সবচেয়ে গুরুত্বপূর্ণ concept।

29.25 First Event Loop Example

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
► CONSOLE OUTPUT OUTPUT

A
C
B
```

কেন?

◆ Step 1

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log("A");
```

Output:

```
► CONSOLE OUTPUT OUTPUT

A
```

◆ Step 2

```
JAVASCRIPT SOURCE JAVASCRIPT

setTimeout( ... )
```

Timer host environment-এ schedule হয়।

◆ Step 3

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log("C");
```

Output:

```
► CONSOLE OUTPUT OUTPUT

C
```

◆ Step 4

Current synchronous code শেষ। তারপর timer callback execute করার সুযোগ পায়।

```
► CONSOLE OUTPUT OUTPUT

B
```

Final:

```
► CONSOLE OUTPUT OUTPUT

A
C
B
```

29.26 setTimeout(... , 0) মানে Immediately নয়

এটা খুব গুরুত্বপূর্ণ।

JAVASCRIPT SOURCE

JAVASCRIPT

```
setTimeout(() => {  
  console.log("Hello");  
}, 0);
```

0 মানে: > যত তাড়াতাড়ি সম্ভব timer-এর callback task হিসেবে schedule করার সুযোগ তৈরি করা। এটা **immediately execute** বোঝায় না। যদি stack-এ blocking কাজ থাকে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);  
  
for (let i = 0; i < 1000000000; i++) {  
  // heavy work  
}
```

তাহলে timer অপেক্ষা করবে।

Part 04 — Microtasks vs Macrotasks: Promises & Execution Order (29.27 – 29.36)

29.27 Microtask vs Task — সবচেয়ে গুরুত্বপূর্ণ Example 🔥🔥

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log("C");  
});  
  
console.log("D");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
A  
D  
C  
B
```

কেন?

♦ প্রথমে synchronous:

► CONSOLE OUTPUT

OUTPUT

```
A  
D
```

তারপর:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Microtask  
↓  
C
```

তারপর:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Task  
↓  
B
```

তাই:

► CONSOLE OUTPUT

OUTPUT

```
A  
D  
C  
B
```

29.28 Execution Diagram

উপরের code:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("A");

setTimeout(() => console.log("B"), 0);

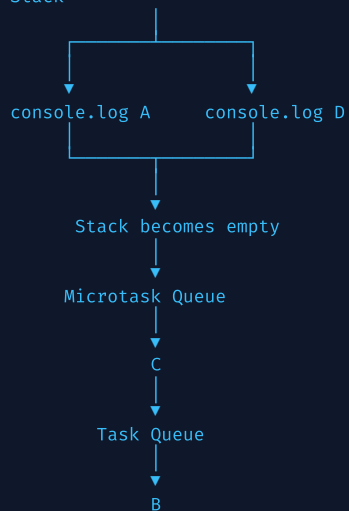
Promise.resolve().then(() => console.log("C"));

console.log("D");
```

Flow:

🕒 ARCHITECTURE / CONCEPT FLOW

Call Stack



Final:

► CONSOLE OUTPUT

OUTPUT

```
A
D
C
B
```

29.29 Promise কেন আগে চলে?

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.resolve().then(() => {
  console.log("Promise");
});

setTimeout(() => {
  console.log("Timer");
}, 0);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Promise
Timer
```

কারণ:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Promise callback → Microtask
Timer callback   → Task
```

Current task শেষ হলে microtasks আগে drain হয়।

29.30 Nested Microtasks

এবার একটু tricky:

```
JAVASCRIPT SOURCE JAVASCRIPT

console.log("A");

Promise.resolve().then(() => {
  console.log("B");

  Promise.resolve().then(() => {
    console.log("C");
  });
});

console.log("D");
```

Output:

```
► CONSOLE OUTPUT OUTPUT

A
D
B
C
```

কারণ:

```
🕒 ARCHITECTURE / CONCEPT FLOW

A
D
↓
Microtask B
↓
B-এর ভিতর C নতুন microtask
↓
C
```

Microtask queue drain হওয়ার সময় নতুন microtask যোগ হলে সেটাও current microtask checkpoint-এ process হতে পারে।

29.31 Timer-এর ভিতরে Promise

```
JAVASCRIPT SOURCE JAVASCRIPT

setTimeout(() => {
  console.log("Timer");

  Promise.resolve().then(() => {
    console.log("Promise");
  });
}, 0);
```

Output:

```
► CONSOLE OUTPUT OUTPUT

Timer
Promise
```

কারণ timer callback একটি task। তার callback-এর synchronous অংশ:

```
► CONSOLE OUTPUT OUTPUT

Timer
```

শেষ হওয়ার পর তার তৈরি করা microtask:

```
► CONSOLE OUTPUT OUTPUT

Promise
```

execute হওয়ার সুযোগ পায়।

29.32 Promise-এর ভিতরে Timer

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.resolve().then(() => {  
  console.log("Promise");  
  
  setTimeout(() => {  
    console.log("Timer");  
  }, 0);  
});
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Promise  
Timer
```

কারণ:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Promise → Microtask  
Timer   → Task
```

29.33 Async/Await এবং Event Loop

`async/await` internally Promise-based behavior-এর সাথে কাজ করে। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function test() {  
  console.log("A");  
  
  await Promise.resolve();  
  
  console.log("B");  
}  
  
test();  
  
console.log("C");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
A  
C  
B
```

কেন? `await` -এর পরে function-এর continuation পরে resume হয়। Conceptually:

🕒 ARCHITECTURE / CONCEPT FLOW

```
test()  
↓  
A  
↓  
await  
↓  
function pauses  
↓  
C  
↓  
microtask  
↓  
B
```

29.34 Async/Await Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function getData() {  
  console.log("Start");  
  
  const result = await Promise.resolve("Data");  
  
  console.log(result);  
}  
  
getData();  
  
console.log("End");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Start  
End  
Data
```

কারণ `await` -এর পরের অংশ asynchronous continuation হিসেবে পরে resume করে।

29.35 `fetch()` এবং Event Loop

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
fetch("/api/users")  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  });
```

Simplified flow:

🕒 ARCHITECTURE / CONCEPT FLOW

```
fetch()  
↓  
Browser handles network operation  
↓  
Response arrives  
↓  
Promise settles  
↓  
.then() callback → Microtask Queue  
↓  
Event Loop  
↓  
Call Stack  
↓  
Callback executes
```

তাই `fetch()` network-এর জন্য JavaScript thread-কে একইভাবে blocking করে বসিয়ে রাখে না।

29.36 DOM Event এবং Event Loop

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

User click করলে browser event handling mechanism callback schedule করে। Simplified:

🕒 ARCHITECTURE / CONCEPT FLOW

```
User clicks button  
↓  
Browser detects event  
↓  
Event callback becomes runnable  
↓  
Task  
↓  
Event Loop  
↓  
Call Stack  
↓  
Callback executes
```

Part 05 — Advanced Runtime Internals, Starvation & Node.js (29.37 – 29.54)

29.37 Blocking Code

এই code:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("Start");  
  
for (let i = 0; i < 1000000000; i++) {  
  // heavy computation  
}  
  
console.log("End");
```

JavaScript main execution thread heavy কাজ করতে থাকলে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Start  
↓  
Heavy Work  
↓  
End
```

এই সময়:

- UI response slow হতে পারে
- click handling delay হতে পারে
- timers delay হতে পারে
- rendering delay হতে পারে
- network result callback execution delay হতে পারে

29.38 Event Loop Magic নয়

অনেকে ভাবে: > JavaScript single-threaded, তাহলে এত asynchronous কাজ কীভাবে করে? উত্তর:

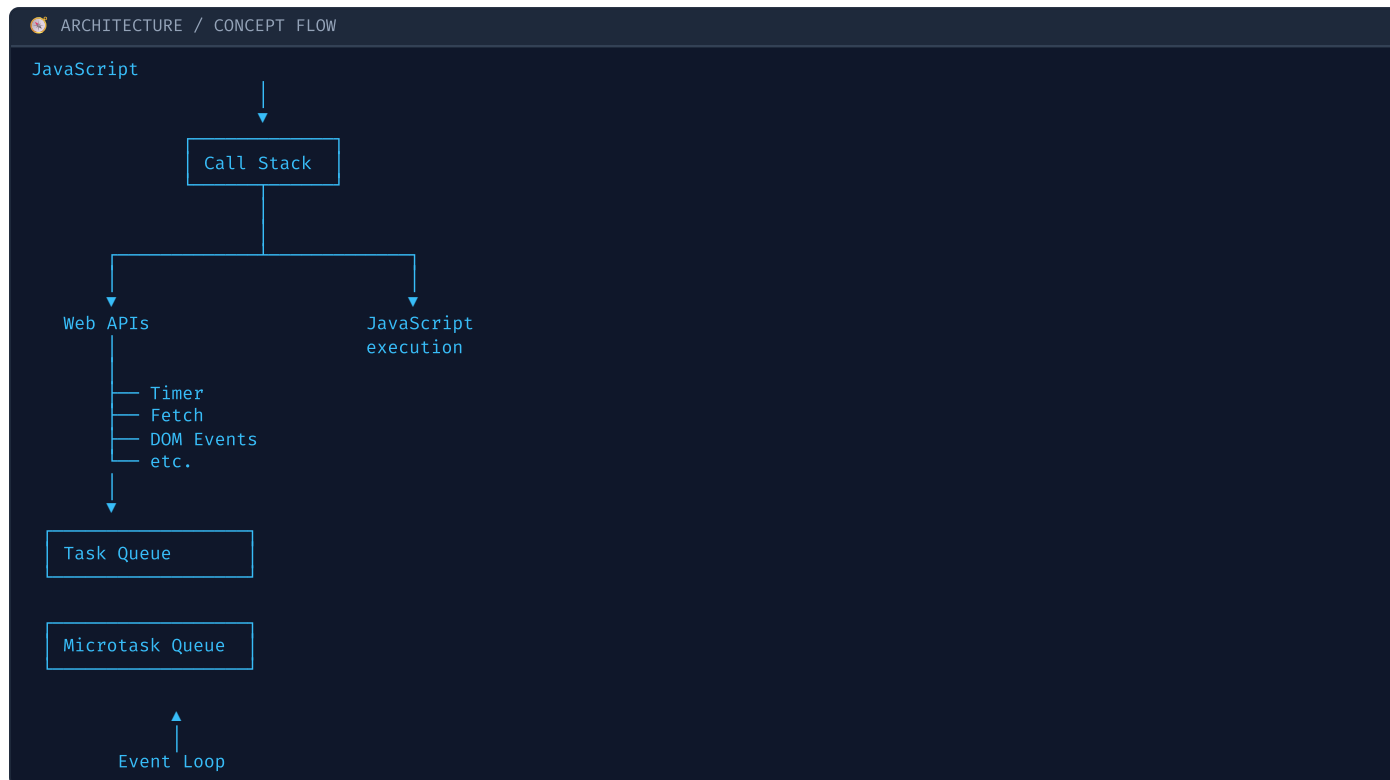
🕒 ARCHITECTURE / CONCEPT FLOW

```
JavaScript  
+  
Host Environment  
+  
Queues  
+  
Event Loop
```

এই combination asynchronous programming সম্ভব করে।

29.39 Browser Runtime Mental Model

একটি simplified diagram মনে রাখো:



29.40 Microtask Queue-এর গুরুত্বপূর্ণ Sources

মনে রাখো:

JAVASCRIPT SOURCE	JAVASCRIPT
<pre>Promise.then() Promise.catch() Promise.finally() queueMicrotask()</pre>	

Browser-এ:

► CONSOLE OUTPUT	OUTPUT
<pre>MutationObserver</pre>	

এগুলো microtask scheduling-এর সাথে সম্পর্কিত।

29.41 Task Queue-এর Common Examples

Common examples:

JAVASCRIPT SOURCE	JAVASCRIPT
<pre>setTimeout() setInterval()</pre>	

এছাড়া browser-এর বিভিন্ন event/task scheduling mechanism আছে। যেমন:

► CONSOLE OUTPUT	OUTPUT
<pre>click keyboard event timer message</pre>	

তবে exact browser scheduling details API/context অনুযায়ী পরিবর্তিত হতে পারে।

29.42 Event Loop-এর Simplified Algorithm

একটি useful mental model:

🕒 ARCHITECTURE / CONCEPT FLOW

```
1. Execute current task
  ↓
2. Call Stack becomes empty
  ↓
3. Drain Microtask Queue
  ↓
4. Browser may perform rendering
  ↓
5. Take next Task
  ↓
6. Repeat
```

সবচেয়ে গুরুত্বপূর্ণ:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Current Task
  ↓
Microtasks
  ↓
Next Task
```

29.43 🔥 Interview Question

Code:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

console.log("4");
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
1
4
3
2
```

♦ কেন?

🕒 ARCHITECTURE / CONCEPT FLOW

```
1 → synchronous
4 → synchronous
3 → microtask
2 → task
```

29.44 আরেকটি গুরুত্বপূর্ণ Question

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise 1");
});

Promise.resolve().then(() => {
  console.log("Promise 2");
});

console.log("End");
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Start
End
Promise 1
Promise 2
Timer
```

কারণ:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Synchronous
↓
Start
End

Microtasks
↓
Promise 1
Promise 2

Task
↓
Timer
```

29.45 আরও Tricky Example 🔥🔥🔥

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("A");

setTimeout(() => {
  console.log("B");

  Promise.resolve().then(() => {
    console.log("C");
  });
}, 0);

Promise.resolve().then(() => {
  console.log("D");

  setTimeout(() => {
    console.log("E");
  }, 0);
});

console.log("F");
```

Expected output:

► CONSOLE OUTPUT

OUTPUT

```
A
F
D
B
C
E
```

Flow:

🕒 ARCHITECTURE / CONCEPT FLOW

```
A
F
↓
Microtask
↓
D
↓
schedule E timer
↓
Next task
↓
B
↓
microtask
↓
C
↓
Next task
↓
E
```

29.46 Microtask Starvation

যদি আমরা continuously microtask তৈরি করতে থাকি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function loop() {
  queueMicrotask(loop);
}

loop();
```

তাহলে microtask queue কখনো empty হওয়ার সুযোগ নাও পেতে পারে। ফলে:

► CONSOLE OUTPUT

OUTPUT

```
Microtasks
Microtasks
Microtasks
Microtasks
...
```

এবং অন্য tasks/rendering অনেক delay হতে পারে। এটাকে broadly: > **Microtask Starvation** বলা হয়।

29.47 কেন Infinite Microtask Dangerous?

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
queueMicrotask(function repeat() {  
  queueMicrotask(repeat);  
});
```

তখন timer:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);
```

প্রত্যাশিত সময়ের মধ্যে execute নাও করতে পারে। কারণ microtask queue repeatedly নতুন কাজ তৈরি করছে।

29.48 Rendering এবং Event Loop

Browser-এ JavaScript execution এবং rendering-এর মধ্যে সম্পর্ক আছে। Heavy synchronous JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
while (true) {  
}
```

করলে browser:

- UI update করতে পারবে না
- user interaction respond করতে পারবে না
- page freeze করতে পারে

তাই frontend development-এ: > **Don't block the main thread unnecessarily.**

29.49 Real-Life Example

ধরো একটি university cafeteria।

♦ JavaScript Main Thread

একজন cashier:

► CONSOLE OUTPUT

OUTPUT

Cashier

একসাথে একজন customer-এর order process করছে।

♦ Host APIs

Kitchen staff:

► CONSOLE OUTPUT

OUTPUT

Kitchen

তারা food প্রস্তুত করছে।

♦ Task Queue

Ready orders-এর queue:

► CONSOLE OUTPUT

OUTPUT

Order 1
Order 2
Order 3

♦ Microtask Queue

Priority ছাড়া follow-up কাজ:

► CONSOLE OUTPUT

OUTPUT

Urgent follow-up

♦ Event Loop

Manager দেখছে:

🕒 ARCHITECTURE / CONCEPT FLOW

Cashier free?
↓
Priority follow-up আছে?
↓
তারপর next order

এটা exact browser implementation নয়, কিন্তু concept বোঝার জন্য useful analogy।

29.50 Node.js-এ Event Loop

Browser আর Node.js-এর host environment এক নয়। Node.js-এ event loop-এর নিজস্ব phases আছে, যেমন:

🕒 ARCHITECTURE / CONCEPT FLOW

Timers
↓
Pending callbacks
↓
Poll
↓
Check
↓
Close callbacks

Node.js-এ আরও গুরুত্বপূর্ণ:

JAVASCRIPT SOURCE

JAVASCRIPT

process.nextTick()

এটা Node-specific এবং সাধারণ Promise microtask-এর সাথে একেবারে একই queue হিসেবে ভাবা উচিত নয়। Node.js event loop আমরা **Node.js-focused Chapter 39**-এ আরও গভীরভাবে দেখব।

29.51 queueMicrotask()

নিজে microtask schedule করতে পারো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("A");

queueMicrotask(() => {
  console.log("B");
});

console.log("C");
```

Output:

► CONSOLE OUTPUT

OUTPUT

A
C
B

কারণ:

🕒 ARCHITECTURE / CONCEPT FLOW

A → sync
C → sync
B → microtask

29.52 Promise.resolve().then() বনাম queueMicrotask()

দুটিই microtask scheduling-এর common way:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.resolve().then(() => {
  console.log("Promise");
});

queueMicrotask(() => {
  console.log("Microtask");
});
```

দুটিই microtask queue-তে callback schedule করে। তবে API behavior/error handling-এর কিছু পার্থক্য আছে। সাধারণ microtask scheduling-এর জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
queueMicrotask()
```

একটি সরাসরি API

29.53 Event Loop + React-এর সম্পর্ক

React development-এর জন্য Event Loop বোঝা খুব usefull যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
button.addEventListener("click", async () => {  
  const response = await fetch("/api/users");  
  
  const data = await response.json();  
  
  console.log(data);  
});
```

Flow:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Click  
↓  
Event callback  
↓  
fetch()  
↓  
Browser handles network  
↓  
Promise settles  
↓  
await continuation  
↓  
Microtask  
↓  
JavaScript continues  
↓  
React state update হতে পারে  
↓  
Browser rendering
```

তাই React-এর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
useEffect()
```

সহ asynchronous programming বুঝতেও Event Loop knowledge কাজে লাগে।

29.54 Common Mistakes

❖ ❌ Mistake 1

`setTimeout(fn, 0)` মানে immediately।

❖ Correct:

▶ CONSOLE OUTPUT

OUTPUT

Minimum delay + task scheduling

❖ ❌ Mistake 2

Promise synchronous।

❖ Correct:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.resolve().then(fn);
```

`fn` asynchronous callback হিসেবে microtask-এ চলে।

❖ ❌ Mistake 3

JavaScript-এর সব asynchronous কাজ JavaScript নিজেই background-এ করে।

❖ Correct:

Host environment/browser/Node.js বিভিন্ন asynchronous operation handle করে।

❖ ❌ Mistake 4

Single-threaded মানে browser কিছুই parallel করতে পারে না।

❖ Correct:

JavaScript execution-এর main thread single-threaded হলেও host environment এবং workers আলাদা execution capabilities দিতে পারে।

❖ ❌ Mistake 5

Microtask এবং Task একই।

❖ Correct:

▶ CONSOLE OUTPUT

OUTPUT

Microtask
Task

দুটির scheduling semantics আলাদা।

29.55 🔥 MUST KNOW

এই Chapter থেকে অবশ্যই জানতে হবে:

♦ Core

- JavaScript execution model
- JavaScript single-threaded concept
- JavaScript engine
- Call Stack
- Heap
- Execution Context
- Global Execution Context
- Function Execution Context
- Creation phase
- Execution phase
- Lexical Environment
- Scope Chain

♦ Event Loop

- Event Loop
- Task Queue
- Microtask Queue
- Web APIs / Host APIs
- `setTimeout`
- Promise callbacks
- `queueMicrotask`
- `async/await`
- `fetch`
- DOM events

♦ সবচেয়ে গুরুত্বপূর্ণ

🕒 ARCHITECTURE / CONCEPT FLOW

Synchronous Code

↓

Call Stack

↓

Microtask Queue

↓

Task Queue

এবং:

🕒 ARCHITECTURE / CONCEPT FLOW

Promise → Microtask

setTimeout → Task

29.Cheat 🧠 Quick Cheat Sheet — Event Loop & Execution Components

JavaScript রানটাইমের প্রতিটি মেকানিজম এবং তার প্রধান ভূমিকার দ্রুত রেফারেন্স টেবিল:

Concept	সহজ অর্থ
Call Stack	কোন JS function এখন চলছে
Heap	Dynamic memory/data-এর সাথে সম্পর্কিত memory area
Execution Context	Code execute করার environment
Global Context	Global code-এর execution environment
Function Context	Function call-এর execution environment
Web API	Browser-provided capabilities
Event Loop	Queue থেকে runnable work execution-এর দিকে আনে
Task Queue	Timer/event ইত্যাদির queued callbacks
Microtask Queue	Promise/queueMicrotask-এর callbacks
Promise callback	Microtask
setTimeout	Task scheduling
await	Promise settle হওয়ার পর continuation resume করে
Stack Overflow	Call Stack অতিরিক্ত পূর্ণ হওয়া
Blocking	Main JS execution আটকে রাখা
Starvation	অন্য কাজ দীর্ঘসময় সুযোগ না পাওয়া

29.Formula 🧮 Output Question Shortcut — ৩ ধাপের গোল্ডেন রুল

যেকোনো জটিল ইন্টারভিউ কোডের এক্সিকিউশন অর্ডার তাৎক্ষণিকভাবে বের করার শটকাট ফর্মুলা:

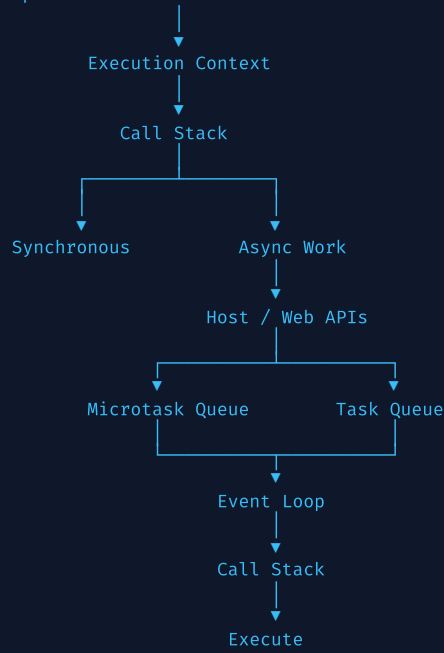
SAMPLE EXECUTION PROBLEM	JAVASCRIPT	GOLDEN EXECUTION PRIORITY
<pre>console.log("A"); setTimeout(() => { console.log("B"); }, 0); Promise.resolve().then(() => { console.log("C"); }); console.log("D");</pre>		1 Synchronous (Immediate) ↓ 2 Microtask Queue (Promises) ↓ 3 Macrotask Queue (Timers)
		▶ CONSOLE OUTPUT OUTPUT A D C B

29.Map 🧠 Final Mental Map & Event Loop Architecture

JavaScript সিংক্রোনাস ও অ্যাসিঙ্ক্রোনাস কোঅর্ডিনেশনের পূর্ণাঙ্গ আর্কিটেকচারাল ফ্লো ম্যাপ:

🕒 EVENT LOOP GLOBAL COORDINATION TREE

JavaScript



★ এক নজরে মূল কথা (Key Takeaway):

JavaScript code প্রথমে Call Stack-এ synchronous-ভাবে execute হয়; asynchronous কাজ host environment-এর মাধ্যমে schedule হয়; Call Stack খালি হলে microtasks আগে process হয়, তারপর tasks-এর সুযোগ আসে—এই coordination-ই Event Loop-এর মূল ভিত্তি।

ইভেন্ট লুপের বিভিন্ন ফেজের অগ্রাধিকার নিখুঁতভাবে যাচাই করতে নিচে ৪টি বহুল জিজ্ঞাসিত চ্যালেঞ্জের সম্পূর্ণ স্টেপ-বাই-স্টেপ ট্রেস সলিউশন দেওয়া হলো:

Practice 1: Timer vs Synchronous Code

BEGINNER

`setTimeout(fn, 0)` বনাম সাধারণ `console.log` এর এক্সিকিউশন অর্ডার যাচাই:

PRACTICE 1 CODE CHALLENGE

JAVASCRIPT

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

▶ CONSOLE OUTPUT

OUTPUT

A
C
B

Step-by-Step Execution Trace:

- 'A' কল স্ট্যাকে তাৎক্ষণিকভাবে এক্সিকিউট হয়ে প্রিন্ট হয়।
- `setTimeout` কল হয়ে টাইমার হোস্ট APIs-এ হ্যান্ডল হয় এবং `oms` পর `Macrotask Queue`-তে যুক্ত হয়।
- 'C' কল স্ট্যাকে এক্সিকিউট হয়ে প্রিন্ট হয়।
- কল স্ট্যাক খালি হলে ইভেন্ট লুপ `Macrotask Queue` থেকে 'B' এনে এক্সিকিউট করে।

Practice 2: Promise Microtask vs Synchronous Code

BEGINNER

`Promise.resolve().then()` বনাম সাধারণ `console.log` এর এক্সিকিউশন অর্ডার:

PRACTICE 2 CODE CHALLENGE

JAVASCRIPT

```
console.log("A");

Promise.resolve().then(() => {
  console.log("B");
});

console.log("C");
```

▶ CONSOLE OUTPUT

OUTPUT

A
C
B

Step-by-Step Execution Trace:

- 'A' সিনক্রোনাসভাবে প্রিন্ট হয়।
- `Promise.resolve()` এর `.then()` কলব্যাক `Microtask Queue`-তে শিডিউল হয়।
- 'C' সিনক্রোনাসভাবে প্রিন্ট হয়।
- সিনক্রোনাস স্ক্রিপ্ট শেষ হওয়ামাত্রই কল স্ট্যাক খালি পেয়ে `Microtask Queue` থেকে 'B' এক্সিকিউট হয়।

Practice 3: Combined Multi-Priority Execution Trace

INTERMEDIATE

Synchronous, Timer ও একাধিক Promise-এর সম্মিলিত এক্সিকিউশন ট্রেস:

PRACTICE 3 CODE CHALLENGEJAVASCRIPT

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

Promise.resolve().then(() => {
  console.log("4");
});

console.log("5");
```

▶ CONSOLE OUTPUTOUTPUT

1
5
3
4
2

 **Step-by-Step Execution Trace:**

১. সিনক্রোনাস কোড: '1' এবং '5' আগে প্রিন্ট হবে।
২. মাইক্রোটাস্ক ভ্রম: '3' এবং '4' মাইক্রোটাস্ক কিউতে থাকায় টাইমার টাস্কের আগে পরপর প্রিন্ট হবে।
৩. ম্যাক্রোটাস্ক এক্সিকিউশন: সব মাইক্রোটাস্ক শেষ হলে ইভেন্ট লুপ টাস্ক কিউ থেকে '2' এক্সিকিউট করবে।

Practice 4: Interleaved Nested Microtasks & Macrotasks

ADVANCED

টাইমারের ভেতর প্রমিজ এবং প্রমিজের ভেতর টাইমারের জটিল ইন্টারলিভড এক্সিকিউশন:

PRACTICE 4 CODE CHALLENGEJAVASCRIPT

```
console.log("A");

setTimeout(() => {
  console.log("B");

  Promise.resolve().then(() => {
    console.log("C");
  });
}, 0);

Promise.resolve().then(() => {
  console.log("D");

  setTimeout(() => {
    console.log("E");
  }, 0);
});

console.log("F");
```

▶ CONSOLE OUTPUTOUTPUT

A
F
D
B
C
E

 **Step-by-Step Execution Trace:**

১. সিনক্রোনাস কোড: 'A' প্রিন্ট হয়, Macrotask Q-তে ঢোকে Timer-B, Microtask Q-তে ঢোকে Promise-D, 'F' প্রিন্ট হয়। [Output: A, F]
২. মাইক্রোটাস্ক ফেজ: Promise-D চলে, 'D' প্রিন্ট করে Macrotask Q-তে Timer-E পাঠায়। [Output: D]
৩. প্রথম ম্যাক্রোটাস্ক: Timer-B চলে, 'B' প্রিন্ট করে Microtask Q-তে Promise-C পাঠায়। [Output: B]
৪. মাইক্রোটাস্ক চেক: পরবর্তী ম্যাক্রোটাস্কে যাওয়ার আগে ইভেন্ট লুপ মাইক্রোটাস্ক Promise-C রান করে। [Output: C]
৫. দ্বিতীয় ম্যাক্রোটাস্ক: Timer-E চলে 'E' প্রিন্ট করে। [Output: E]
চূড়ান্ত রেজাল্ট: A → F → D → B → C → E

 **Next Chapter Preview:**

Chapter 30 — JavaScript Memory Management & Garbage Collection

পরবর্তী চ্যাপ্টারে আমরা শিখব Memory Lifecycle, Memory Allocation, Stack vs Heap, Reference Counting, Mark-and-Sweep Garbage Collection এবং Memory Leak প্রতিকার।